Visualização Geométrica, Rasterização e Recorte de Árvores Arteriais CCO

Trabalho Prático 1 – BCC327 Computação Gráfica

Heitor Augusto de Oliveira Ramos Rafael Stylita Duarte

Departamento de Computação (DECOM)
Universidade Federal de Ouro Preto (UFOP)

2026.2

Roteiro da Apresentação

1. Contexto e Objetivos
2. Dados e Metodologia
3. Implementação
4. Resultados
5. Discussão e Conclusão

Contexto do Trabalho

Problema

Uma árvore CCO é só um conjunto de números: pontos, pares de índices e um raio por vaso. Para virar imagem, ela precisa passar por todas as etapas do *pipeline* gráfico.

- Modelos fornecidos em formato `.vtk` (ASCII, `POLYDATA`).
- Uma árvore 2D e uma 3D, ambas com 64 terminais.
- Cada vaso é um segmento entre dois pontos, com um atributo `raio`.
- Foco: entender e implementar os algoritmos clássicos, não só chamar a biblioteca.

Objetivos

Objetivo Geral

Uma aplicação única que lê, representa, transforma, rasteriza e recorta os modelos CCO, separando o que foi implementado pelo grupo do que vem da biblioteca.

- M1: leitura, primitivas e cor
- M2: transformações, câmera e projeções
- M3: Bresenham
- M4: Scanline Fill
- M5: aliasing e antialiasing
- M6: Cohen–Sutherland e Liang–Barsky
- M7: Sutherland–Hodgman

Dados Utilizados

- `tree2D_Nterm064.vtk` e `tree3D_Nterm064.vtk`.
- Estrutura:
  - `POINTS`: coordenadas (x, y, z);
  - `LINES`: `2 id_início id_fim`;
  - `CELL_DATA`: escalar `raio` por segmento.
- Leitor próprio, sem o leitor de VTK da biblioteca.

Modelos analisados

	2D	**3D**
Pontos	128	128
Segmentos	127	127
Raio mínimo	0,2032	0,2052
Raio máximo	1,1814	1,0178
r_{max}/r_{min}	5,81	4,96

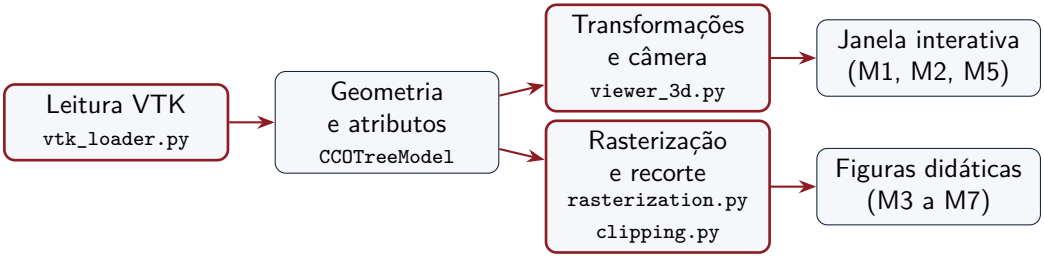
Tecnologias Utilizadas

- Linguagem: Python 3.12.
- Biblioteca gráfica: PyVista 0.43.3 sobre VTK 9.3.0 (OpenGL).
- NumPy 1.26.4 para matrizes; Matplotlib 3.8.3 só para desenhar as figuras didáticas.
- Saída: visualizador interativo e figuras PNG para relatório e slides.

Ambiente de execução

`make install` cria o `venv` e instala as dependências fixadas; `make run-3d`, `make run-2d`, `make plots` e `make screenshots` executam a aplicação. O Makefile funciona em Linux/macOS e Windows.

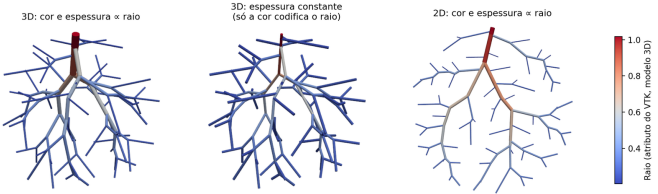
Fluxo Geral da Aplicação



Borda vermelha: algoritmos implementados pelo grupo. Um único `main.py` com três modos: `viewer`, `plots` e `screenshots`.

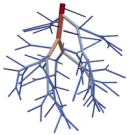
Representação Geométrica e Cores

- Cada vaso é um **tubo próprio**, com 2 pontos exclusivos: o raio não é interpolado entre vasos vizinhos.
- Espessura $\propto$ raio do arquivo.
- Mapa contínuo `coolwarm` (azul = fino, vermelho = grosso): o raio é contínuo, e uma paleta RGB discreta criaria fronteiras artificiais.
- Tecla `C`: espessura constante, só a cor codifica o raio.

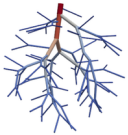


Transformações Geométricas

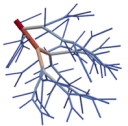
Original (vista lateral, câmera fixa)



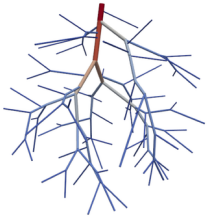
Translação em Y (4 passos de 5% da diagonal)



Rotação de 45° em X (pivô no centroide)



Escala uniforme ×1.61 (centroide fixo)

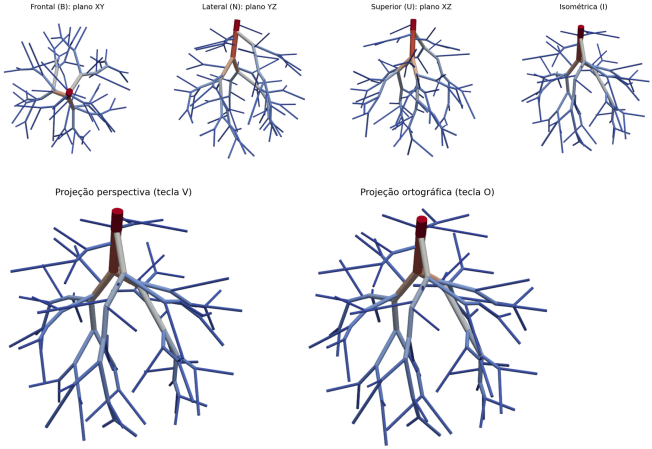


- Matrizes homogêneas 4×4 em NumPy: $T(\mathbf{d})$, $R_x, R_y, R_z(\theta)$, $S(s)$.
- Em torno do centroide: $M' = T(\mathbf{c})\, M\, T(-\mathbf{c})$.
- Acúmulo $M_{acc} \leftarrow M'\, M_{acc}$, impresso no terminal.

- Teclas: `T/G` translação, `K/J` rotação de 15°, `+/-` escala, `0` reinicia.
- Câmera fixa: a mudança na imagem vem só das coordenadas no mundo.

Câmera, Vistas e Projeções

- **Mundo**: coordenadas do modelo após as transformações.
- **Câmera**: base $(\mathbf{u}, \mathbf{v}, \mathbf{w})$ a partir de posição, foco e *up*; o observador fica na origem.
- Vistas: `B` frontal, `N` lateral, `U` superior, `I` isométrica.
- `O` ortográfica: tamanho não depende da profundidade.
- `V` perspectiva: $x_p = d\, x_c/(-z_c)$.

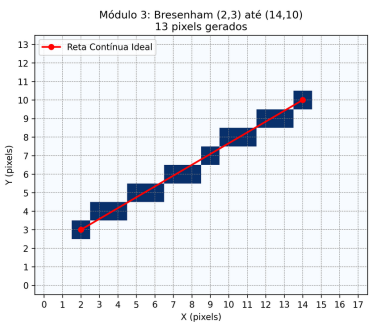


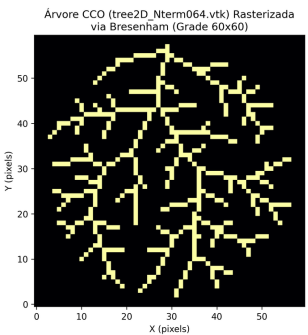
Técnicas de Computação Gráfica Implementadas

Técnica	**Como foi utilizada no projeto**	**Origem**
Transformações	matrizes 4×4 de translação, rotação e escala com pivô	grupo
Câmera / projeções	4 vistas; ortográfica $\times$ perspectiva	PyVista
Rasterização de linhas	Bresenham nos 8 octantes, aritmética inteira	grupo
Rasterização de polígonos	Scanline Fill com ET e AET, regra par-ímpar	grupo
Antialiasing	supersampling $k \times k$ por cobertura; MSAA/SSAA no viewer	grupo / VTK
Recorte de segmentos	Cohen–Sutherland e Liang–Barsky (Cyrus–Beck discutido)	grupo
Recorte de polígonos	Sutherland–Hodgman (Weiler–Atherton discutido)	grupo

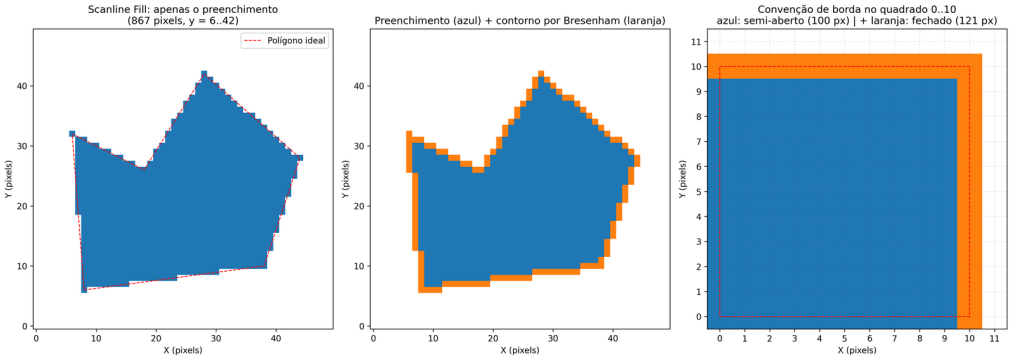
Módulo 3: Bresenham

- $(2, 3) \rightarrow (14, 10)$: $\Delta x = 12$, $\Delta y = 7$.
- $e = \Delta x - \Delta y = 5$; a cada passo $e_2 = 2e$:
  - $e_2 > -\Delta y$: avança x;
  - $e_2 < \Delta x$: avança y.
- $\max(12, 7) + 1 = 13$ pixels, só com somas inteiras.
- À direita: os 127 vasos da árvore 2D numa grade 60×60.



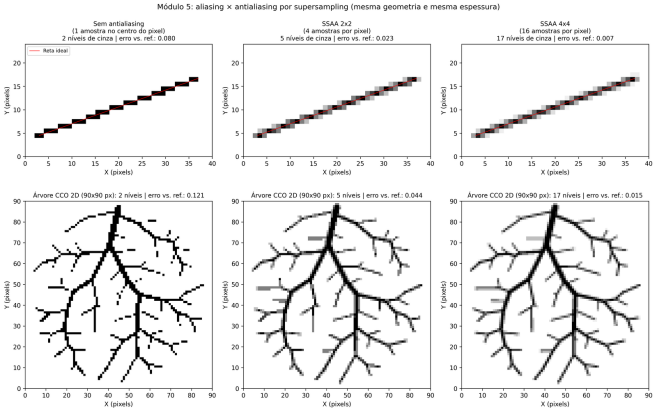


Módulo 4: Scanline Fill



- Polígono côncavo de 6 vértices: 867 pixels nas linhas $y = 6..42$.
- Arestas ativas em $[y_{min}, y_{max})$: vértices compartilhados não são contados duas vezes.
- Quadrado 0..10: 121 pixels (fechado) $\times$ 100 pixels (semiaberto, sem sobreposição entre vizinhos).

Módulo 5: Aliasing × Antialiasing



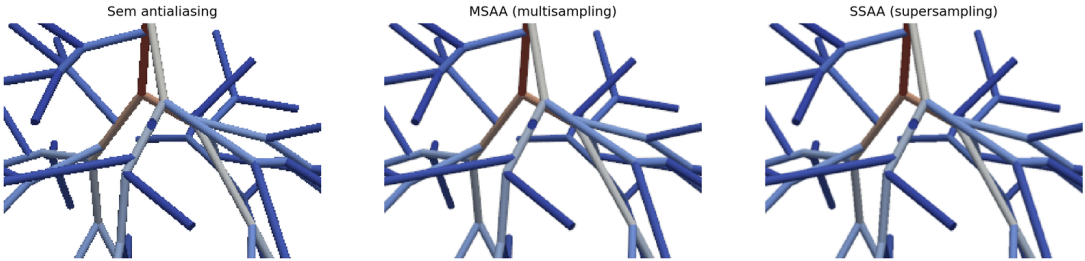
Mesma geometria e espessura; só muda o número de amostras por pixel.

Amostras	Níveis	Erro reta	Erro árvore
1	2	0,080	0,121
2×2	5	0,023	0,044
4×4	17	0,007	0,015

- Sem AA: escada na reta e vasos finos fragmentados.
- 4×4: erro $\approx 11\times$ menor, custo $16\times$ maior.

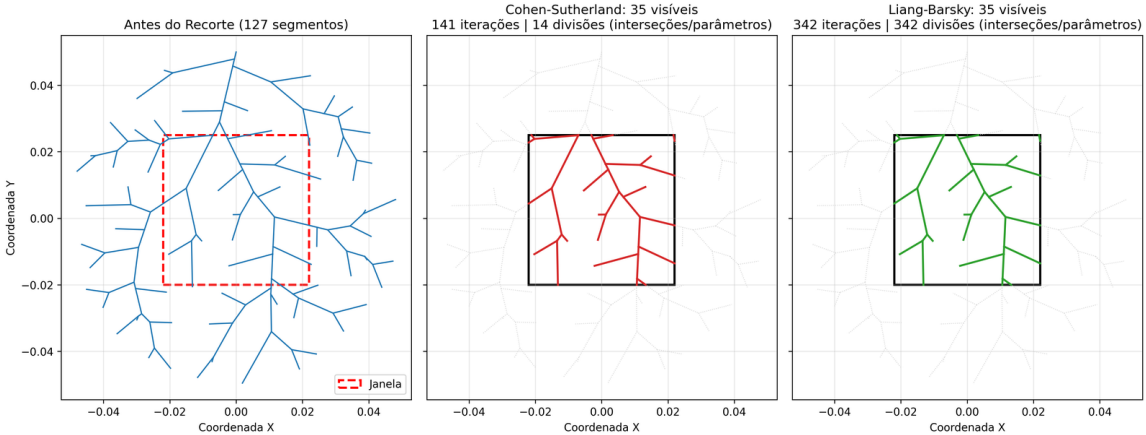
Módulo 5: Antialiasing no Visualizador

Antialiasing no visualizador (tecla A): recorte ampliado da mesma região



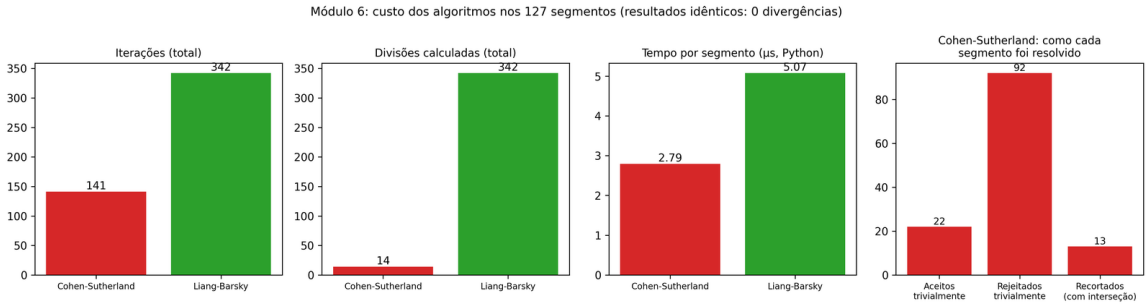
- Tecla `A`: desligado → MSAA → SSAA (fornecidos pelo VTK).
- MSAA: várias amostras de cobertura por pixel, cor calculada uma vez por primitiva.
- SSAA: renderiza em resolução maior e reduz, mais caro e trata também o sombreamento.

Módulo 6: Recorte de Segmentos



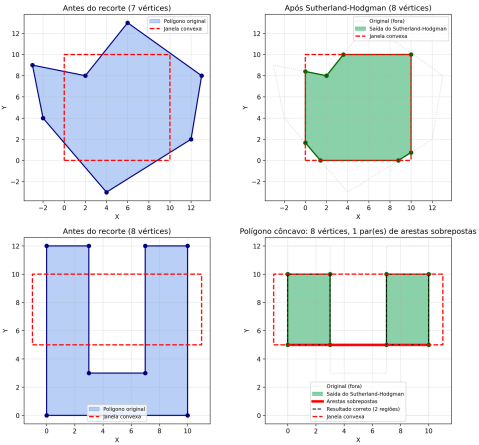
Janela $[-0{,}022;\ 0{,}022] \times [-0{,}020;\ 0{,}025]$: **35 segmentos visíveis nos dois algoritmos, 0 divergências**.

Módulo 6: Comparação de Custo



- **Cohen–Sutherland**: 114 de 127 segmentos resolvidos só com códigos de região; 14 interseções.
- **Liang–Barsky**: sempre 4 testes de borda (342 divisões), mas calcula cada extremo uma única vez. Vantajoso quando muitos segmentos cruzam a janela.
- **Cyrus–Beck**: mesma ideia paramétrica para janelas *convexas quaisquer*, usando normais $\mathbf{N}_i$; Liang–Barsky é o caso retangular.

Módulo 7: Recorte de Polígonos



Sutherland–Hodgman

Recorta contra cada aresta da janela convexa (4 casos dentro/fora). Janela validada e reorientada para anti-horário.

- Caso 1: 7 vértices → 8 vértices, resultado correto.
- Caso "U": o correto são 2 regiões, mas a saída é 1 lista de 8 vértices com **arestas sobrepostas** em $y = 5$.

Quando usar Weiler–Atherton

Polígonos côncavos que se dividem em várias regiões, janelas côncavas, buracos e operações booleanas: percorre as fronteiras alternando nas interseções e devolve várias regiões.

Recurso Adicional

Medição e reprodutibilidade dos resultados

Os algoritmos de recorte contam iterações, divisões e casos triviais (`ClipStats`); o antialiasing mede níveis de cinza e erro contra uma referência 16×16; o modo `screenshots` regera as capturas do visualizador sem abrir janela.

- **Motivação:** comparar algoritmos com números, e não só visualmente.
- **Como:** contadores opcionais nas funções de recorte; `CCOViewer` em modo *off-screen* acionado pelos mesmos métodos das teclas.
- **Resultado:** todas as figuras e tabelas do relatório são regeradas com `make plots` e `make screenshots`.

Discussão

Implementado pelo grupo

Leitor VTK; matrizes 4×4 e pivô; tubos por segmento; Bresenham; Scanline (ET/AET); supersampling; Cohen–Sutherland; Liang–Barsky; Sutherland–Hodgman.

Com auxílio de biblioteca

PyVista/VTK: janela, câmera, projeções, malha dos tubos, mapa de cores, iluminação, MSAA/SSAA. Matplotlib: só exibição.

- **Dificuldades:** raio interpolado entre vasos (resolvido com pontos duplicados); vértices compartilhados no Scanline; comparação justa de antialiasing.
- **Limitações:** recorte e rasterização didáticos no plano XY; malha reconstruída a cada transformação; Weiler–Atherton e Cyrus–Beck só discutidos.
- **Melhorias:** matriz aplicada na GPU, esferas nas bifurcações, Weiler–Atherton.

Conclusões

- Os sete módulos do TP1 foram atendidos sobre os modelos CCO 2D e 3D.
- Supersampling 4×4 reduziu o erro de cobertura em cerca de 11 vezes.
- Cohen–Sutherland e Liang–Barsky: mesmo recorte, custos diferentes conforme a cena.
- O caso côncavo mostra na prática a limitação do Sutherland–Hodgman.
- Base pronta para o TP2: Z-buffer, iluminação, transparência e *ray casting*.

Mensagem final

Cada pixel da árvore CCO na tela passou por algoritmos que o grupo implementou ou consegue explicar, da leitura do VTK ao recorte.

Referências

- FOLEY, J. D. et al. *Computer Graphics: Principles and Practice*. Addison-Wesley, 1996.
- HEARN, D.; BAKER, M. *Computer Graphics with OpenGL*. Pearson, 2011.
- ANGEL, E.; SHREINER, D. *Interactive Computer Graphics*. Pearson, 2015.
- SCHREINER, W.; BUXBAUM, P. Computer-optimization of vascular trees. *IEEE Transactions on Biomedical Engineering*, 1993.
- LIANG, Y.-D.; BARSKY, B. A. A new concept and method for line clipping. *ACM TOG*, 1984.
- SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. *CACM*, 1974.
- VTK Documentation. `https://vtk.org/` PyVista. `https://docs.pyvista.org/`

Comandos de Execução

```
make install          # cria o venv e instala as dependencias
make run-3d           # visualizador interativo, modelo 3D
make run-2d           # visualizador interativo, modelo 2D
make plots            # figuras dos modulos 3 a 7
make screenshots      # capturas do visualizador (modulos 1, 2 e 5)
make clean            # remove venv e __pycache__
```

Observação

O README traz dependências e versões, teclas do visualizador e um roteiro mínimo de teste.

Obrigado!

Perguntas?

Heitor Augusto de Oliveira Ramos ▪ Rafael Stylita Duarte